Prefetching Using Principles of Hippocampal-Neocortical Interaction

Michael Wu
Yale University
mw976@yale.edu

Ketaki Joshi
Yale University
ketaki.joshi@yale.edu

Andrew Sheinberg
Yale University
andrew.sheinberg@gmail.com

Guilherme Cox
NVIDIA
gcox@nvidia.com

Anurag Khandelwal
Yale University
anurag.khandelwal@yale.edu

Raghavendra Pradyumna Pothukuchi
Yale University
raghav.pothukuchi@yale.edu

Abhishek Bhattacharjee
Yale University
abhishek.bhattacharjee@yale.edu

Abstract

Memory prefetching improves performance across many systems layers. However, achieving a reasonable tradeoff between accuracy and overhead is challenging as memory hierarchies become more complicated. The prefetcher's ability to adjust when new access patterns emerge is also crucial. Recent work has demonstrated the use of deep learning techniques to improve prefetching accuracy, albeit with impractical compute and storage overheads. This paper suggests taking inspiration from the learning mechanisms and memory architecture of the human brain — specifically the hippocampus and neocortex — to build resource-efficient, accurate, and adaptable prefetchers.

ACM Reference Format:
Michael Wu, Ketaki Joshi, Andrew Sheinberg, Guilherme Cox, Anurag Khandelwal, Raghavendra Pradyumna Pothukuchi, and Abhishek Bhattacharjee. 2023. Prefetching Using Principles of Hippocampal-Neocortical Interaction. In *Workshop on Hot Topics in Operating Systems (HotOS '23), June 22–24, 2023, Providence, RI, USA.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3593856.3595901

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
HotOS '23, June 22–24, 2023, Providence, RI, USA
© 2023 Association for Computing Machinery.
ACM ISBN 979-8-4007-0195-5/23/06…$15.00
https://doi.org/10.1145/3593856.3595901

1 Introduction

Memory prefetching is a performance optimization used widely across several hardware and software layers of modern computer systems. Prefetching proactively brings data from slower levels of memory to faster ones, anticipating its future use. Although well-studied, prefetching continues to be explored, especially as emerging memory hierarchies embrace heterogeneity [20], disaggregation [25], vertical/horizontal tiering [29], compute in memory [45], and more.

Early prefetchers targeted patterns that were easy to capture, such as strides, and were sufficient for well-understood applications, such as those in SPEC [4]. However, systems and applications today are far more complex and dynamic, rendering simple approaches ineffective. There is a growing interest in developing prefetchers that can adapt to the dynamic execution by *learning* memory access patterns instead of detecting pre-programmed rules [11, 16, 37] .

Recent studies have begun exploring the viability of deep learning (DL) for prefetching. In theory, DL should improve prefetching because it is inherently data-driven, and should naturally adapt to workloads and their environments. These studies show that, in idealized simulations, DL outperforms non-learning approaches in prefetch accuracy [16, 28, 37]. However, all of these approaches have three main shortcomings that impede their real-world adoption.

First, the deep neural networks (DNNs) in prior work use impractically high compute and storage resources, sometimes on the order of running an entirely separate application (Sec. 2.1). Realizing a more efficient and lightweight learning structure is crucial to real-world integration in systems software and hardware.

Second, most prior approaches train their models offline. This was a necessity due to the overhead of DNN training.

However, the models can then fail to adapt to evolving workload phases, configurations, input, and other system dynamics. Offline training also requires labeled datasets, which can be impractical to collect at scale.

Third, even if performance optimizations advanced to solve issues of resource usage, online DNNs would still face the challenge of catastrophic interference. This issue is well known in machine learning (ML) literature [21, 26], and describes the tendency of DNNs to drastically forget previously learned information when learning new information.

We consider the fundamental challenge of how to accurately learn memory access patterns online and without interference. In this paper, we argue that we can develop a robust solution by incorporating principles of learning found in the human brain. We note that the problem of learning and predicting access patterns is similar to a problem that the human brain encounters continuously, which is to discover generalizable structure from the experiences presented to it.

In particular, we believe the cognitive theory of Complementary Learning Systems (CLS) [30], which models the human brain as an *online* learning system, offers important inspiration for creating *efficient* and *online* prefetchers. CLS theory posits that the brain avoids catastrophic interference using *interleaved replay*. When encountering new information, the brain records it into the hippocampus instead of immediately updating its decision-making network, the neocortex. Over time, old and new information are interleaved into the neocortex, learning both without interference.

Many principles from CLS have already seen use in the ML community [13, 14, 41, 46]. Their objectives and constraints, however, differ from ours. In addition to maximizing model accuracy, we must further account for metrics like training and inference latency, prefetch timeliness, and usage of memory/network bandwidth. The combination of these constraints motivates a more domain-specific solution.

As we discuss how CLS principles can improve upon DL approaches, we also present expected implementation challenges with two real contexts in mind: disaggregated systems [25] and CPU-GPU systems [3]. We specifically choose these because they experience high cross-node communication latency and operate in relatively resource-constrained settings. Thus, they stand to greatly benefit from intelligent memory prefetching. Additionally, they provide the opportunity to implement CLS principles in software, offering a faster path to immediate impact in production systems.

2 Limitations of Prior Learning Approaches

Prior approaches using DL for prefetching make use of deep neural networks (DNNs). DNNs are characterized by the use of multiple layers of neurons, with all-to-all connections between layers. In prefetching, the specific DNN models used are often transformers [28, 42] or Long Short-Term Memories (LSTMs) [16, 18, 37]. These models are trained to predict the cache lines or memory pages an application will use given recent accesses and instructions, and often achieve high accuracy. However, they have unrealistic resource overheads, and thus remain only simulated ideals. Some works have used more lightweight learning algorithms for systems, such as reinforcement learning [11], gradient boosting [38], and small DNNs [22]. Still, these approaches were only evaluated in simulation, and their small models would fail to match the accuracy achievable with DNNs. Since we envision maximizing prefetching accuracy, we restrict our focus to the barriers in deploying DL approaches.

2.1 Resource Limitations

Prior DNN prefetching models incur untenable compute/storage overheads. A state-of-the-art LSTM based prefetcher required over 1GB of storage using 32-bit parameters [37]. This already exceeds the memory capacity of some of our target systems [25, 36]. Thus, for the sake of realistic comparison, we create a reduced version of the model sized less than 1MB. We then measure the CPU latency for inference and training[1]. We measure CPU performance because our target inference times are between 1-10μs [7, 25, 36], making it unclear whether accelerator offloading would outweigh offload costs. Fig. 1 shows the performance results: the LSTM model takes >150μs per inference and >1ms per example for training, making it impractical for real-world deployments.

We also measure latency after quantizing the model. Quantization reduces the bit-width of model parameters (e.g. fp32 to int8). This reduces storage and compute overheads, but makes the model less amenable for training [27]. As such, it is primarily an inference optimization for offline-trained models. Even after quantization, the inferency latency of the LSTM is still much higher than our target at >60μs.

2.2 Issues with Online Learning

To effectively train a DNN offline, the training data must accurately represent the test scenarios. However, the scale of modern systems and dynamism of their workloads can make collecting representative datasets impossible. Ideally, a model would instead learn to adapt system behavior online. This, however, exposes the model to catastrophic interference.

2.2.1 Catastrophic Interference is triggered when a model trained on one task switches to a different, unrelated task, forgetting the first task in the process. To illustrate this, we run experiments using the LSTM from (2.1). The model is first trained on only one memory access pattern (*i.e.,* strides), simulating online training on a single program phase. Once it attains perfect accuracy on this pattern, it is swapped to train on a different pattern only (*i.e.,* pointer chasing).

[1]Models are compiled using PyTorch C++ libraries [5].

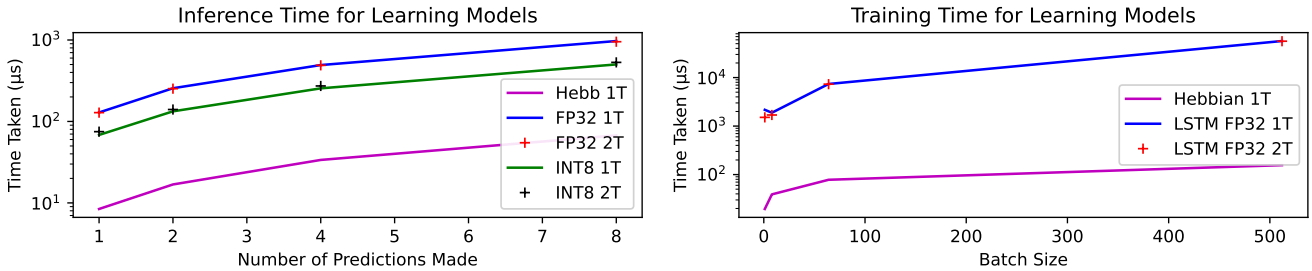


Figure 1: Inference and training latency on an Intel i7-8700 CPU. Note that LSTMs are notoriously difficult to parallelize [39]. This, along with the small model size, explains why multithreading does not improve performance.

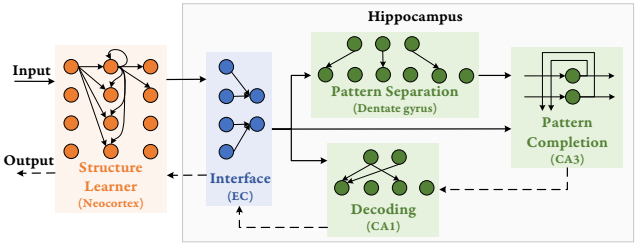


Figure 2: Memory architecture of the brain in CLS theory. Each block is modeled with a Hebbian neural network, which is significantly more lightweight than a DNN. Solid lines show information storage paths and dashed lines show recall and replay. Replay storage and sampling occur in the brain's CA3 region.

To observe the effects of interference, we measure the model's confidence in predicting each pattern correctly as a function of iterations spent training the second pattern. Confidence is to the probability the network assigns to a prediction. Figures 3a-3c show our results. The red lines indicates confidence in the first pattern. A value of 100% indicates the model is always correct with full confidence. As the second pattern is trained (blue), the first pattern confidence drops to 0%. This means the model will be incorrect with absolute certainty, indicating catastrophic interference. In a real system, this would make the model ineffective for programs with repeated phases.

One could argue that catastrophic interference could be avoided by periodically snapshotting the model. However, this further inflates the storage overheads of DL. Further, choosing when to restore a snapshot and which to restore is non-trivial, perhaps needing its own learning algorithm. A real implementation requires a more practical solution.

3 CLS-Inspired Prefetching

There are significant learning and resource-oriented barriers to implementing DNN-based prefetchers in real systems. CLS theory provides important insights for overcoming both.

Figure 2 shows the brain's memory architecture in CLS theory. At a high level, memory is managed through a complementary relationship between two regions of the brain: the neocortex and hippocampus. The neocortex slowly learns the structure underlying the information it encounters (analogous to the rules behind a memory access pattern) similar to DNNs. Meanwhile, the hippocampus quickly memorizes the information it encounters (such as the actual memory accesses) in a compressed format (through pattern separation). Over time, this information is decompressed and replayed in the neocortex, interleaving the learning of old and new tasks. This *replay* process (analogous to data shuffling in offline training) mitigates interference between old and new information [23, 30].

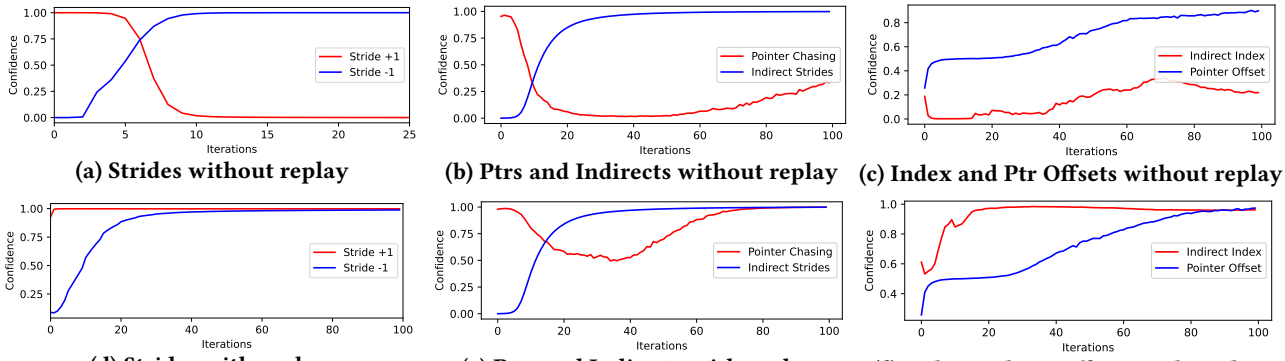
CLS also analyzes the behavior of biologically-inspired neural networks, which require significantly less compute and storage than DNNs and, even with idealized no-cost DNN, remain comparable in prediction accuracy (3.2).

3.1 Mitigating Catastrophic Interference

Catastrophic interference occurs when a model is switched to learning a disjoint task. In CLS theory, this issue is mitigated by *replaying* past memories stored in the hippocampus. Note that replay is also used in reinforcement learning [14]. However, common implementations assume an unlimited buffer for storing replay examples. Building a hippocampal storage for an in-system network is an open problem, and will require insights on how to selectively store and sample scenarios. In this section, we show the benefits of replay for prefetching without resource considerations.

3.1.1 Candidate Access Patterns: We demonstrate replay for a suite of memory access patterns (Table 1). Our list (adapted from prior work [10]) represents patterns a universal prefetcher should detect based on common code constructs. For each experiment, we generate a trace of 1000 accesses for two patterns and observe their interactions in the model.

3.1.2 Replay Experiments: For each selected pair of patterns, we use the same base training methodology from 2.2.1. To introduce replay, we retrain on the first pattern at $\frac{1}{10}^{\text{th}}$ the learning rate after each iteration of the second. Figure 3 shows our results. Models trained without replay clearly show dramatic drops in confidence for the first pattern. In some cases, the first pattern confidence recovers partially, indicating transferrable knowledge. With replay however,



(a) Strides without replay (b) Ptrs and Indirects without replay (c) Index and Ptr Offsets without replay

(d) Strides with replay (e) Ptrs and Indirects with replay (f) Index and Ptr Offsets with replay

Figure 3: The effect of replay on interference between two memory access patterns in a neural network. Confidence in the first pattern is shown in red, while confidence in the second is in blue.

Pattern	Behavior	Code
Stride	Accessing data at regular delta such as streaming patterns or array traversal.	a[i]
Pointer Chase	Pseudorandom accesses to different parts of the memory. E.g. linked list traversal	*ptr
Indirect Stride	Accessing data at regular delta from a base address. E.g. array of object pointers.	*(a[i])
Indirect Index	Accessing data at indices that are at regular delta from a constant base address. E.g. array of indices	b[a[i]]
Pointer Offset	Pseudorandom accesses and adjacent data. E.g. transform on a list/tree.	*ptr *(ptr+i)

Table 1: Memory Access Patterns

confidence in the original pattern fully recovers. In this scenario, the network would maintain prediction accuracy if the program repeats its previous phase.

For the sake of demonstration, we've naively stored all past examples for replay, and assume they can all be interleaved later. A real implementation must tradeoff the storage/compute costs with the learning benefits of replay. Moreover, our preliminary studies show only one form of replay. Cognitive literature describes many forms of replay [43], each with their own benefits and challenges. We discuss possible approaches to implementing replay in 4.5.

3.2 Overcoming Resource Constraints

CLS theory primarily uses sparse Hebbian networks for learning. The meaning of sparse is twofold. First, the networks are sparse in *connectivity*. Unlike all-to-all connections in DNNs, a node in a CLS Hebbian net connects to only 1-25% of the nodes in adjacent layers. In addition, they are sparse in their *representations*. That is, when inputs pass through the network, only 1-25% of middle layer neurons are non-zero.

The immediate effect of sparsity is significantly reduced storage and compute requirements. Compared to DNNs, only a fraction of the weights are stored, and only a fraction is iterated over for computation. This difference is further compounded by using the Hebbian update rule:

$$\Delta w_{ij} = (b_j \neq 0)[(a_i \neq 0) - (a_i == 0)]$$

Model	Parameters	Ops (inference)	Ops (training)
LSTM	170k	> 170k FPU	> 400k FPU
Hebbian	49k	14k ALU	64k ALU

Table 2: Resource Costs of Learning Models

Here, a weight (w_{ij}) increases if both neurons it connects are non-zero, and decreases if the input neuron (a_i) is inactive while the output (b_j) is active. This update requires far fewer ALU/FLOPS than the conventional loss functions of DNNs.

We now demonstrate the compute and prediction performance of a prototype sparse Hebbian neural network. Our prototype uses a single hidden layer of 1000 neurons, with 12.5% connectivity to the input and output layers. We set the sparsity of the hidden representation to 10% by setting the 100 highest-valued neurons to 1 and the others to 0.

Table 2 shows the resource usage of our Hebbian network. Note that without 12.5% sparsity, our network would be over double the size of the LSTM from 2.2. Instead, it is $\sim 3x$ smaller. Additionally, compute costs are reduced by an order of magnitude. This is exemplified in Fig. 1, which shows compute latency on an Intel i7-8700 CPU. In both cases, the Hebbian model is at least an order of magnitude faster: 8μs vs 60μs for inference, and 20μs vs 1000μs for training.

Although Hebbian networks significantly reduce compute overhead, they must not reduce prediction accuracy, even versus an idealized (and hence unrealizable) DNN. We thus evaluate the ability of each model to perform page prefetching on the following workloads: TensorFlow [6] training ResNet-50 [17], PageRank [32] run using GraphChi [24], mcf [4], and graph500 [1]. We select these workloads because they demonstrate the access patterns from Table 1.

For each workload, we collect a trace of 2 billion memory accesses and simulate them with a memory sized at 50% of the trace's footprint. We use a simple scheme for training and prediction. Training examples are generated by storing the address of the last page miss. On each fault, the prefetcher is trained to predict the current miss given the last, and also makes a new prediction. This constitutes a simple scheme for defining the prefetcher's task and generating training data. We discuss ideas of more sophisticated approaches in 4.2.

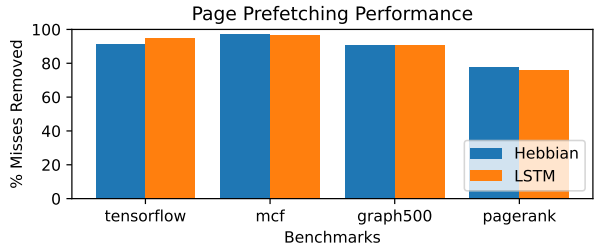


Figure 4: Page prefetching performance of Hebbian and LSTM models

Figure 4 shows our results. We measure the percentage of misses removed compared to a baseline without prefetching. Our prototype model has comparable accuracy to a state-of-the-art DNN, all while consuming far fewer resources.

4 Future Research Challenges

Although our initial results are promising, harnessing the benefits of CLS principles will require solving several challenges pertaining to real-world deployment. Many of these challenges involve complex interactions between the accuracy of learning models and the systems surrounding them. Thus, this section discusses these challenges with a focus on our two target environments: disaggregated systems [25] and CPU-GPU accelerator systems [3]. Both systems (Fig. 5) orchestrate data movement from a central module, and stand to benefit from prefetching due to high communication latencies [7, 25]. They differ, however, in how page misses can affect performance. In disaggregated systems, CPUs fault on one page at a time, indicating a prefetcher should be optimized to hide latency. In GPUs, SIMT execution can produce many concurrent faults, and lockstep execution causes a single fault to stall multiple threads. This indicates the need for a more throughput-optimized prefetcher. As we introduce the challenges to deploying a CLS-inspired prefetcher, we will also indicate how solutions may change when optimizing for throughput or latency.

4.1 Online Training

The first challenge to tackle is how to handle the overheads of training a neural network online. Our simulated experiments performed a training iteration on every miss, but this may be unrealistic in practice. In particular, because training is more expensive than inference, it may be important to explore asynchronous approaches and batching. These help hide training latency, but can introduce delayed learning effects where the model does not learn in time for local behavior. This would be very application-specific, and we expect further study would be needed.

It is also important to ensure that model training does not contend for resources with applications. This motivates sampling approaches. Sampling reduces overall training costs, but presents potential information issues. It's possible that simple sampling techniques (e.g. random) may miss cases that are critical for the model to understand the workload. A more intelligent sampling process could use confidence measures from the model (perhaps from early layers) to filter probable noise, or to avoid training on well-learned cases.

After handling the costs of online training, it is also important to consider how training may affect inference availability. Specifically, since training actively changes the weights of a neural network, it may be important to block inference during training. This is an availability issue, which motivates a protocol where training is applied to a separate model copy, which is later redeployed when the live model's confidence/accuracy decreases. However, it is possible that simpler approaches could also suffice. First, because prefetching is not correctness-critical, inference requests are safe to drop. Perhaps a more interesting insight, however, is that neural networks can have noise-robust or noise-smoothing effects [19, 33]. This could allow inference to remain accurate even when concurrent with training. We expect this property to require further study, as it could significantly optimize a real prefetcher deployment, but may be sensitive to many details of the training implementation.

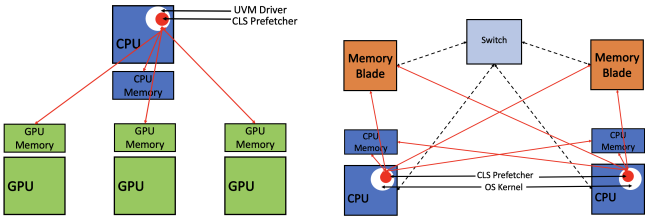
4.2 Generating Training Data

After handling the challenges of online training, there still remains the issue of generating useful training data. The training data given to a model fundamentally determines the tasks that can be learned. If the data does not imply a useful task, even a perfectly accurate network will be useless. Take our scheme from 3.2, where the network is trained to always predict the next miss. If the time between misses is less than the inference latency, then even a perfect model will always prefetch too late. Here, a more effective method could store a window of past misses, allowing a prefetcher to be trained to fetch misses further in the future.

More generally, the best data generation method is dependent on the parameters of the prefetcher and the surrounding system. A higher-latency prefetcher must learn to prefetch further into the future. Workloads with locally predictable but globally uncorrelated behavior may benefit more from timestamp-based miss tracking over a fixed window size. Throughput-bound systems may benefit more from a less accurate prefetcher that can predict multiple options at a time. The same could be said for read-heavy workloads, as eviction will less expensive. Network-bottlenecked systems, however, would require the opposite. This all indicates the need for codesign between the prefetcher, system, and data generation process.

4.3 Encoding Data for Neural Networks

After generation, input and target data must be intelligently encoded for a neural network. Most prior work encodes addresses and strides as one-hot vectors, which then index an embedding table for a dense representation [16, 28, 37]. This



(a) CLS - UVM driver **(b) CLS - MIND system**

Figure 5: Architectures of our target systems. Red lines indicate logical paths of prefetch requests, while dashed lines in (b) indicate network connections.

aligns with approaches used for words in natural language processing, but storing embeddings can become expensive (>500MB in [37]). It also inflates compute costs, as the size of the output layer grows linearly with the number of embedding vectors.

A more fundamental issue, though, is that addresses and strides are often a poor proxy for understanding the inherent behavior of an application. We've found that, as of now, neither the LSTM nor Hebbian networks perform well on caching workloads (memcached [15], cachebench [12]). This is due to these workloads being almost entirely pointer-based, with little to no linear patterns. Thus, learning access patterns is reduced to memorizing exact sequences of addresses or strides, an inefficient process for neural networks.

Ideally, the representation of the input data would more closely resemble how addresses "flow" at the data structure level (e.g. tree nodes, pointer chains). We have found that cognitive ideas can provide inspiration here as well. There is evidence that hippocampal "grid cells" encode location information that optimizes for vector-based navigation [31]. A similar encoding used for addresses could better represent paths through different data structures. Other cognitively-inspired work has explored ways to "bind" symbolic information to allow the efficient detection of relations in neural networks [8]. An analog in prefetching would be an address embedding optimized for detecting pointers that are logically (as opposed to numerically) close to one another.

4.4 Model Placement

After the designing the prefetching model and its periphery, one must then decide exactly where to place the model in the system. In general, the placement of the prefetcher(s) will be a function of the resources needed to use information, as well as the potential for optimizations. A more global prefetcher will have access to more information, but will also require more processing to ensure that different information streams do not interfere. This could require more sophisticated encodings, neural network parameters, peripheral processing, etc. This scheme is shown in Fig. 5a, where the prefetcher is placed in the CPU driver, and can separate streams of information by which node the originate from. A decentralized scheme is shown in Fig. 5b, where each CPU core has its own prefetcher. This is partially necessitated due to resource scarcity on the network switch, but also provides benefits in reducing interference. It is also more amenable to microarchitectual optimizations, such as cache allocation technology [2].

These examples are non-exhaustive. There is a large space of exploration in model placement. Further examples include having separate prefetchers per application for scenarios where avoiding interference is the priority, or running models on the GPU to tradeoff latency and resource contention for throughput. We expect determining the best prefetcher placement for any given system will require significant study of these learning and performance tradeoffs.

4.5 Implementing Replay

Building a replay mechanism introduces a number of implementation concerns. First, there is the question of the size of the replay storage. A larger storage will demand more resources, but it will better ensure that the most important data from earlier phases is kept. Naturally, the next question is what to store for replay. An intelligent mechanism for selectively storing examples could reduce resource requirements while maintaining replay quality. This could be accomplished by filtering unnecessary samples utilizing neural storage structures, which provide similarity scores between inputs and contents [19, 33]. When to replay is the final question. It will be crucial to perform replay when CPU utilization is low, so as to minimize competition with client applications.

In a SIMT model, it is possible that information will naturally be interleaved as a result of many concurrent threads. This could provide no-cost replay, but may be unpredictable due the non-transparent GPU hardware scheduler. It is possible that this could be made consistent by exposing areas of the hardware scheduler to software.

5 Conclusion

Recent deep learning approaches to prefetching have shown promising results, but only in idealized simulations. Their real implementation is impeded due to resource overheads and learning limitations. This paper explores how one might address these issues using principles and models of the hippocampus and neocortex, as well as some challenges we should expect in implementing them. We ultimately expect more challenges to appear in designing and deploying such models, but we hope this paper serves as a starting point for implementing efficient and intelligent learning algorithms for all relevant systems contexts.

References

[1] 2010. Graph 500 | large-scale benchmarks. https://graph500.org/.

[2] 2016. Introduction to Cache Allocation Technology in the Intel® Xeon® Processor E5 v4 Family. https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-cache-allocation-technology.html.

[3] 2017. Nvidia Unified Memory. https://developer.nvidia.com/blog/unified-memory-cuda-beginners/.

[4] 2017. SPEC Benchmarks and Tools. https://www.spec.org/benchmarks.html.

[5] 2018. libtorch. https://github.com/pytorch/pytorch/blob/master/docs/libtorch.rst.

[6] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. 2016. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467* (2016).

[7] Tyler Allen and Rong Ge. 2021. Demystifying GPU UVM Cost with Deep Runtime and Workload Analysis. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 141–150. https://doi.org/10.1109/IPDPS49936.2021.00023

[8] Awni Altabaa, Taylor Webb, Jonathan Cohen, and John Lafferty. 2023. Abstractors: Transformer Modules for Symbolic Message Passing and Relational Reasoning. *arXiv preprint arXiv:2304.00195* (2023).

[9] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. 2012. Workload Analysis of a Large-Scale Key-Value Store. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems* (London, England, UK) *(SIGMETRICS '12)*. Association for Computing Machinery, New York, NY, USA, 53–64. https://doi.org/10.1145/2254756.2254766

[10] Grant Ayers, Heiner Litz, Christos Kozyrakis, and Parthasarathy Ranganathan. 2020. Classifying Memory Access Patterns for Prefetching. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 513–526. https://doi.org/10.1145/3373376.3378498

[11] Rahul Bera, Konstantinos Kanellopoulos, Anant Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. 2021. Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 1121–1137. https://doi.org/10.1145/3466752.3480114

[12] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosof, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. 2020. The CacheLib Caching Engine: Design and Experiences at Scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 753–768. https://www.usenix.org/conference/osdi20/presentation/berg

[13] Prabuddha Chakraborty and Swarup Bhunia. 2022. BINGO: brain-inspired learning memory. *Neural Computing and Applications* 34 (02 2022), 1–25. https://doi.org/10.1007/s00521-021-06484-8

[14] William Fedus, Prajit Ramachandran, Rishabh Agarwal, Yoshua Bengio, Hugo Larochelle, Mark Rowland, and Will Dabney. 2020. Revisiting Fundamentals of Experience Replay. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)*. JMLR.org, Article 287, 11 pages.

[15] Brad Fitzpatrick. 2004. Distributed caching with memcached. *Linux Journal* (August 2004).

[16] Milad Hashemi, Kevin Swersky, Jamie Smith, Grant Ayers, Heiner Litz, Jichuan Chang, Christos Kozyrakis, and Parthasarathy Ranganathan. 2018. Learning Memory Access Patterns. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 1919–1928.

[17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[18] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (nov 1997), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[19] Heiko Hoffmann. 2019. Sparse Associative Memory. *Neural Computation* 31, 5 (05 2019), 998–1014. https://doi.org/10.1162/neco_a_01181 arXiv:https://direct.mit.edu/neco/article-pdf/31/5/998/1052590/neco_a_01181.pdf

[20] Sudarsun Kannan, Ada Gavrilovska, Vishal Gupta, and Karsten Schwan. 2017. HeteroOS: OS Design for Heterogeneous Memory Management in Datacenter. *SIGARCH Comput. Archit. News* 45, 2 (jun 2017), 521–534. https://doi.org/10.1145/3140659.3080245

[21] Ronald Kemker, Marc McClure, Angelina Abitino, Tyler Hayes, and Christopher Kanan. 2018. Measuring catastrophic forgetting in neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.

[22] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 489–504. https://doi.org/10.1145/3183713.3196909

[23] Dharshan Kumaran, Demis Hassabis, and James L McClelland. 2016. What Learning Systems do Intelligent Agents Need? Complementary Learning Systems Theory Updated. *Trends in Cognitive Sciences* 20, 7 (2016), 512–524. https://doi.org/10.1016/j.tics.2016.05.004

[24] Aapo Kyrola, Guy Blelloch, and Carlos Guestrin. 2012. GraphChi: Large-Scale Graph Computation on Just a PC. In *Proceedings of the 10th USENIX Conference on Operating Systems Design and Implementation* (Hollywood, CA, USA) *(OSDI'12)*. USENIX Association, USA, 31–46.

[25] Seung-seob Lee, Yanpeng Yu, Yupeng Tang, Anurag Khandelwal, Lin Zhong, and Abhishek Bhattacharjee. 2021. MIND: In-Network Memory Management for Disaggregated Data Centers. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) *(SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 488–504. https://doi.org/10.1145/3477132.3483561

[26] Sang-Woo Lee, Jin-Hwa Kim, Jaehyun Jun, Jung-Woo Ha, and Byoung-Tak Zhang. 2017. Overcoming catastrophic forgetting by incremental moment matching. *Advances in neural information processing systems* 30 (2017).

[27] Hao Li, Soham De, Zheng Xu, Christoph Studer, Hanan Samet, and Tom Goldstein. 2017. Training Quantized Nets: A Deeper Understanding. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/1c303b0eed3133200cf715285011b4e4-Paper.pdf

[28] Xinjian Long, Xiangyang Gong, Bo Zhang, and Huiyang Zhou. 2023. Deep learning based data prefetching in CPU-GPU unified virtual memory. *J. Parallel and Distrib. Comput.* 174 (Apr 2023), 19–31. https://doi.org/10.1016/j.jpdc.2022.12.004

[29] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. 2022. Tpp: Transparent page placement for cxl-enabled tiered memory. *arXiv preprint*

arXiv:2206.02878 (2022).

[30] James L McClelland, Bruce L McNaughton, and Randall C O'Reilly. 1995. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological review* 102, 3 (1995), 419.

[31] B. L. McNAUGHTON, C. A. Barnes, J. L. Gerrard, K. Gothard, M. W. Jung, J. J. Knierim, H. Kudrimoti, Y. Qin, W. E. Skaggs, M. Suster, and K. L. Weaver. 1996. Deciphering The Hippocampal Polyglot: the Hippocampus as a Path Integration System. *Journal of Experimental Biology* 199, 1 (01 1996), 173–185. https://doi.org/10.1242/jeb.199.1.173 arXiv:https://journals.biologists.com/jeb/article-pdf/199/1/173/2532105/jexbio_199_1_173.pdf

[32] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank citation ranking: Bringing order to the web.* Technical Report. Stanford InfoLab.

[33] Edmund Rolls. 2018. The storage and recall of memories in the hippocampo-cortical system. *Cell and Tissue Research* 373 (09 2018). https://doi.org/10.1007/s00441-017-2744-3

[34] Geet Sethi, Bilge Acun, Niket Agarwal, Christos Kozyrakis, Caroline Trippel, and Carole-Jean Wu. 2022. RecShard: Statistical Feature-Based Memory Optimization for Industry-Scale Neural Recommendation. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 344–358. https://doi.org/10.1145/3503222.3507777

[35] Mohammad Shahrad, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference (USENIX ATC'20)*. USENIX Association, USA, Article 14, 14 pages.

[36] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. 2018. Legoos: A disseminated, distributed {OS} for hardware resource disaggregation. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*. 69–87.

[37] Zhan Shi, Akanksha Jain, Kevin Swersky, Milad Hashemi, Parthasarathy Ranganathan, and Calvin Lin. 2021. A Hierarchical Neural Model of Data Prefetching. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) *(ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 861–873. https://doi.org/10.1145/3445814.3446752

[38] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. 2020. Learning Relaxed Belady for Content Distribution Network Caching. In *Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation* (Santa Clara, CA, USA) *(NSDI'20)*. USENIX Association, USA, 529–544.

[39] Marijn F. Stollenga, Wonmin Byeon, Marcus Liwicki, and Juergen Schmidhuber. 2015. Parallel Multi-Dimensional LSTM, with Application to Fast Biomedical Volumetric Image Segmentation. In *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2* (Montreal, Canada) *(NIPS'15)*. MIT Press, Cambridge, MA, USA, 2998–3006.

[40] Elvira Teran, Zhe Wang, and Daniel A. Jiménez. 2016. Perceptron learning for reuse prediction. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12. https://doi.org/10.1109/MICRO.2016.7783705

[41] Simon Thomann, Paul R. Genssler, and Hussam Amrouch. 2022. HW/SW Co-design for Reliable In-memory Brain-inspired Hyperdimensional Computing. https://doi.org/10.48550/ARXIV.2202.04789

[42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[43] Lennart Wittkuhn, Samson Chien, Sam Hall-McMaster, and Nicolas W. Schuck. 2021. Replay in minds and machines. *Neuroscience & Biobehavioral Reviews* 129 (2021), 367–388. https://doi.org/10.1016/j.neubiorev.2021.08.002

[44] Juncheng Yang, Yao Yue, and K. V. Rashmi. 2021. A Large-Scale Analysis of Hundreds of In-Memory Key-Value Cache Clusters at Twitter. *ACM Trans. Storage* 17, 3, Article 17 (aug 2021), 35 pages. https://doi.org/10.1145/3468521

[45] Shimeng Yu, Wonbo Shim, Xiaochen Peng, and Yandong Luo. 2021. RRAM for compute-in-memory: From inference to training. *IEEE Transactions on Circuits and Systems I: Regular Papers* 68, 7 (2021), 2753–2765.

[46] Xuejiao Zhao, Huanhuan Chen, Zhenchang Xing, and Chunyan Miao. 2020. Brain-inspired Search Engine Assistant based on Knowledge Graph. https://doi.org/10.48550/ARXIV.2012.13529